Deadlock Conclusion.

Dr. Jit Mukherjee

Recovery From Deadlock

- A deadlock detection algo determines a deadlock exists
  - Inform the operator that deadlock occurs. Deal with manually
  - Recover from deadlock automatically.
- Process Termination
  - Abort all the deadlock process - clearly break but great expenses
  - Abort one process at a time until deadlock breaks - considerable overhead, after one process is aborted check if deadlock still exists
- Midst updating a file, printing -> Reset
- Partial termination -> which deadlocked process to terminate
  - Policy decision like CPU scheduling
  - Abort those processes -> Minimum cost. Defining minimum cost
- Minimum Cost
  - Priority
  - Exhausted CPU burst an remaining CPU bursts
  - How many and what type of resources -> can be preempte?

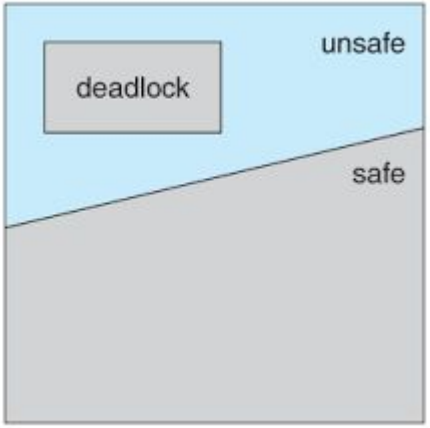
Recovery from Deadlock

- Process Termination - Minimum Cost
  - How many more resources are required
  - How many processes need to be terminated
  - Whether process is interactive or batch
- Resource Preemption - Preempt some resource from P_i and give to P_j until deadlock is broken.
- Issues -
  - Selecting a Victim - Which resources and which processes should be preempted.
    - Minimum Cost - Number of resources a process is holding, how much time left
  - Rollback - Roll back the process to safe state.
    - Difficult to comprehend safe state
    - Total Rollback - Needs to keep more information about states of all processes
  - Starvation - How can we ensure starvation will not occur.
    - Victim Selection based on cost factor
    - A process can be victim for finite number of times

Deadlock Avoidance

- Deadlock prevention - Lo device utilizations and reduced system throughput.
- Additional information - How resources are requested.
  - A system with 1 tape drive, 1 printer, in which order a process is requesting and releasing and how another process is behaving.
  - Complete sequence of requests and releases
- Resources
  - Currently available
  - Currently allocated
  - Future requests and releases
- Simplest - Each process declare max number of each resource it needs
- A deadlock avoidance algo - Dynamically examines resource allocation state to prevent circular wait
  - Resource allocation state - number of available and allocated resources, max demand

Safe State



- A state is safe if the system can allocate resources
  - To each process up to its maximum
  - Still avoid a deadlock
- Safe sequence <P_0, P_1, ...,P_n> - for current allocation state if, for each P_i, the resource request of P_i, can be satisfied by the currently available resources + resources held by P_j.
  - If the resources P_i need - not available, P_i waits until all P_j complete.
  - When they have finished P_i can obtain all the resources
  - After P_i, P_(i+1) can complete.
  - If no such sequence exits, the system is unsafe.
- An unsafe state may lead to deadlock. Not all unsafe state - deadlocked

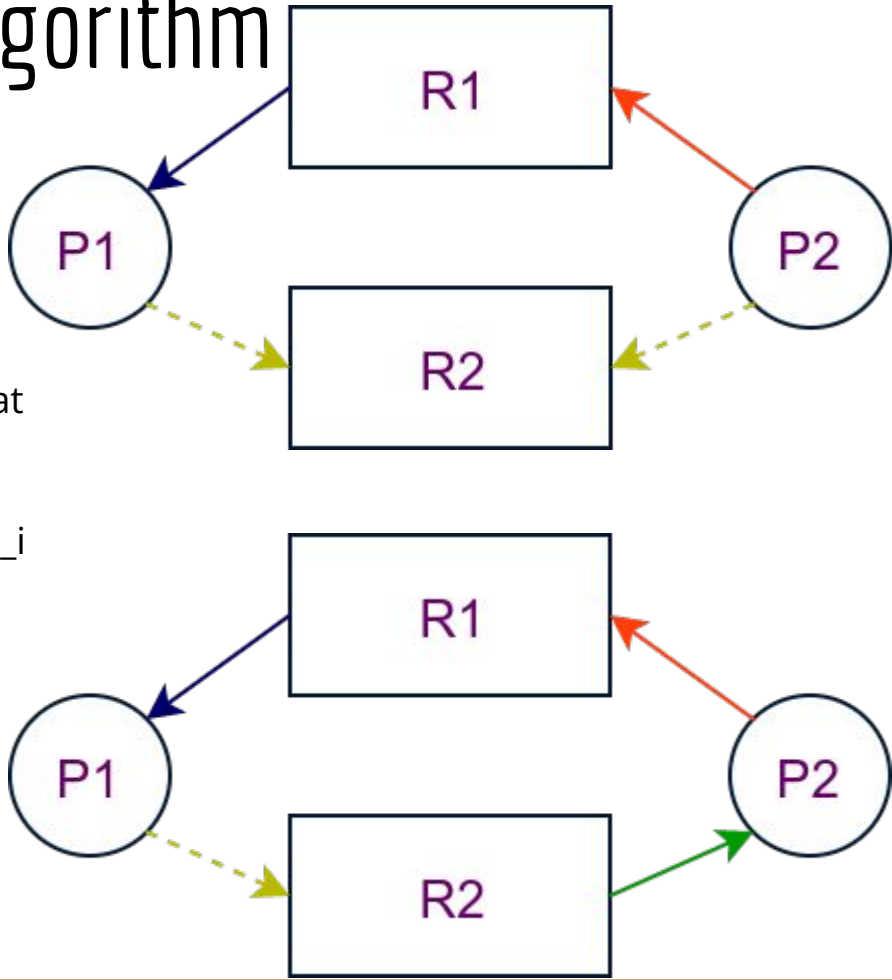
Safe State

- 12 printer, 3 process
- P_0 requires max 10 printers, holding 5 printers
- P_1 requires max 4 printers, holding 2 printers
- P_2 requires max 9 printers, holding 2 printers
- Free printers at t_0 - 3
- The sequence <P_1, P_0, P_2> satisfies the safety condition.
- It can go to unsafe state - at t_1 P_2 gets another resource. It becomes unsafe
  - P_1 gets all and completes
  - Available 4 printers
  - Does not satisfy P_0, P_2

	Max Need	Current Need
P_0	10	5
P_1	4	2
P_2	9	2

Resource Allocation Graph Algorithm

- All resource -> **One** instance
- A variant of RA graph -> Deadlock avoidance
  - Claim Edge - P_i - -> R_j -> P_i may request R_j at t_k
  - P_i requests R_j, claim edge to request edge
  - R_j is released by P_i, assignment edge R_j -> P_i => P_i - -> R_j
  - Request edge direction but dashed line
- The resources must be **claimed** at t_0
- P_i -> R_j => R_j -> P_i
  - If it does not forma cycle
  - Cycle detection. Time complexity of n^2, n = number of vertices



Banker's Algorithm

- It is a banker algorithm used to **avoid deadlock** and **allocate resources** safely to each process in the computer system.
- The '**S-State'** examines all possible tests or activities before deciding whether the allocation should be allowed to each process.
    - Whether a person should be sanctioned a loan amount or not to help the bank system safely simulate allocation resources.
    - number of account holders = 'n', and the total money = 'T'. If an account holder applies for a loan; first, the bank subtracts the loan amount from full cash and then estimates the cash difference is greater than T to approve the loan amount.
    - if another person applies for a loan or withdraws some amount from the bank, it helps the bank manage
- How much each process can request for each resource in the system. It is denoted by the [**MAX**] request.
- How much each process is currently holding each resource in a system. It is denoted by the [**ALLOCATED**] resource.
- It represents the number of each resource currently available in the system. It is denoted by the [**AVAILABLE**] resource.

Banker’s Algorithm

n is the number of processes, and m is the number of each type of resource

1. **Available**: It is an array of length 'm' that defines each type of resource available in the system. When Available[j] = K, means that 'K' instances of Resources type R[j] are available in the system.
2. **Max:** It is a [n x m] matrix that indicates each process P[i] can store the maximum number of resources R[j] (each type) in a system.
3. **Allocation:** It is a matrix of m x n orders that indicates the type of resources currently allocated to each process in the system. When Allocation [i, j] = K, it means that process P[i] is currently allocated K instances of Resources type R[j] in the system.
4. **Need:** It is an M x N matrix sequence representing the number of remaining resources for each process. When the Need[i] [j] = k, then process P[i] may require K more instances of resources type Rj to complete the assigned work. Nedd[i][j] = Max[i][j] - Allocation[i][j].
5. **Finish**: It is the vector of the order **m**. It includes a Boolean value (true/false) indicating whether the process has been allocated to the requested resources, and all resources have been released after finishing its task.

The Banker's Algorithm is the combination of the safety algorithm and the resource request algorithm to control the processes and avoid deadlock in a system

Banker’s Algorithm: Safety Algorithm

It is a safety algorithm used to check whether or not a system is in a safe state or follows the safe sequence in a banker's algorithm:

1. There are two vectors **Wok** and **Finish** of length m and n in a safety algorithm.
    a. Initialize: Work = Available
    b. Finish[i] = false; for I = 0, 1, 2, 3, 4... n - 1.
2. Check the availability status for each type of resources [i], such as:
    a. Need[i] <= Work
    b. Finish[i] == false
    c. If the i does not exist, go to step 4.
3. Work = Work +Allocation(i) // to get new resource allocation
    a. Finish[i] = true
    b. Go to step 2 to check the status of resource availability for the next process.
4. If Finish[i] == true; it means that the system is safe for all processes.

- Disadvantage -
    - It requires a fixed number of processes, and no additional processes can be started in the system while executing the process.
    - The algorithm does no longer allows the processes to exchange its maximum needs while processing its tasks.
    - Each process has to know and state their maximum resource requirement in advance for the system.
    - The number of resource requests can be granted in a finite time, but the time limit for allocating the resources is one year.

Banker's Algorithm: Resource Request Algorithm

A resource request algorithm checks how a system will behave when a process makes each type of resource request in a system as a request matrix.Let create a resource request array R[i] for each process P[i]. If the Resource Request [j] equal to 'K', which means the process P[i] requires 'k' instances of Resources type R[j] in the system.

1. When the number of **requested resources** of each type is less than the **Need** resources, go to step 2 and if the condition fails, which means that the process P[i] exceeds its maximum claim for the resource. As the expression suggests:
    a. If Request(i) <= Need
    b. Go to step 2;
2. When the number of requested resources of each type is less than the available resource for each process, go to step (3). As the expression suggests:
    a. If Request(i) <= Available
    b. Else Process P[i] must wait for the resource since it is not available for use.
3. When the requested resource is allocated to the process by changing state:
    a. Available = Available - Request
    b. Allocation(i) = Allocation(i) + Request (i)
    c. $Need_i = Need_i - Request_i$

When the resource allocation state is safe, its resources are allocated to the process P(i). And if the new state is unsafe, the Process P (i) has to wait for each type of Request R(i) and restore the old resource-allocation state.

Example

Context of the need matrix is as follows:

Need [i] = Max [i] - Allocation [i]

- Need for
  - P1: (7, 5, 3) - (0, 1, 0) = 7, 4, 3
  - P2: (3, 2, 2) - (2, 0, 0) = 1, 2, 2
  - P3: (9, 0, 2) - (3, 0, 2) = 6, 0, 0
  - P4: (2, 2, 2) - (2, 1, 1) = 0, 1, 1
  - P5: (4, 3, 3) - (0, 0, 2) = 4, 3, 1
- For P1: Need <= Available -> False.
- P2: Need <= Available -> True.
  - New available = available + Allocation
  - (3, 3, 2) + (2, 0, 0) => 5, 3, 2.
- P3: Need <= Available False.
- P4: Need <= Available.
  - 0, 1, 1 <= 5, 3, 2 condition is true.
  - 5, 3, 2 + 2, 1, 1 => 7, 4, 3. P5
- P5: 4, 3, 1 <= 7, 4, 3 condition is **true**
  - 7, 4, 3 + 0, 0, 2 => 7, 4, 5 -> New Available
- Repeat for P1 and P3

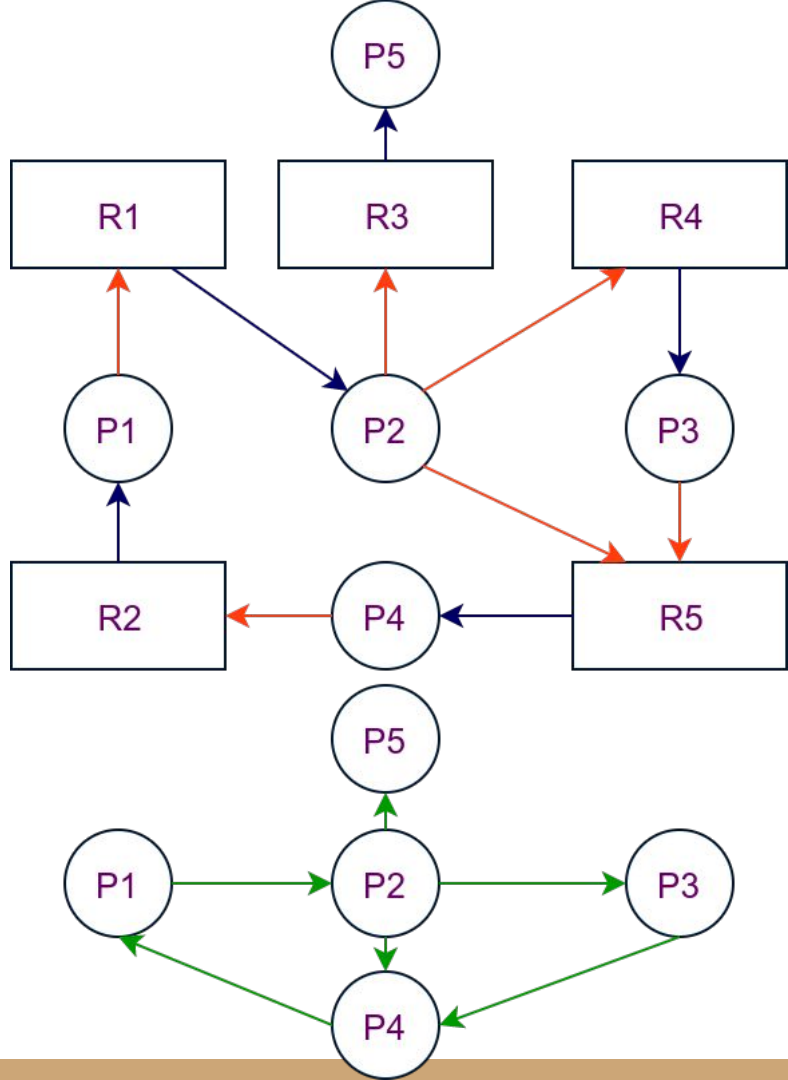
Safe Sequence - **P2, P4, P5, P1 and P3.**

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P1	0	1	0	7	5	3	3	3	2
P2	2	0	0	3	2	2			
P3	3	0	2	9	0	2			
P4	2	1	1	2	2	2			
P5	0	0	2	4	3	3			

Process	Need		
	A	B	C
P1	7	4	3
P2	1	2	2
P3	6	0	0
P4	0	1	1
P5	4	3	1

Deadlock Detection

- No deadlock prevention or avoidance
  - An algorithm that examines -> Whether deadlock has happened.
  - An algorithm to recover.
- **Single Instance of Each Resource Type**
- Wait-for Graph - Variant of RA graph
  - By removing the resource nodes and collapsing appropriate edges
- P_i -> P_j => P_i is waiting for P_j to complete/release a resource P_i needs
- P_i -> P_j exists if and only if P_i -> R_k and R_k -> P_j
- A deadlock exists if and only if cycle in wait-for graph.
- Maintain wait-for graph an periodically check.



Several Instances of A Resource

- The algorithm employs several time-varying data structures:
  - **Available**. A vector of length m indicates the number of available resources of each type.
  - **Allocation**. An n x m matrix defines the number of resources of each type currently allocated to each process.
  - **Request**. An n x m matrix indicates the current request of each process.
    - If Request[i][j] = k, P_i is requesting k more instances of R_j
- Row in Allocation -> Allocation_i, Request -> Request_i
- Investigate every possible allocation sequence for all P_i to be completed

1. Let Work and Finish be vector of m and n length. Work = Available.
   a. For i = 0,1, .. n-1 if Allocation != 0 -> Finish[i] = false
   b. Else Finish[i] = true
2. Find an index i -> Finish[i] == false and Request_i <= Work. If no go to 4
3. Work = Work + Allocation_i
   a. Finish[i] = true
   b. Go to Step 2
4. If Finish[i] == false, for i -> Deadlock -> P_i is deadlocked

Several Instances of A Resource

- In step 3 - reclaim the resource of P_i when Request_i <= Work.
  - We know P_i is not in a deadlock (Request_i <= Work)
- Optimistic -> P_i requires no resources
  - If our assumption incorrect -> Next Detection algo
- First table no deadlock -> <P_0, P_2, P_3, P_1, P_4 >
- Second table P_2 additional request
  - Deadlock
  - Though P_0 will execute. <P_1, P_2, P_3, P_4> -> Deadlock

	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	3	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

	Request		
	A	B	C
P0	0	0	0
P1	2	0	2
P2	0	0	1
P3	1	0	0
P4	0	0	2